



Security Audit Report

Findings

Authors: Darko Deuric, Marko Juric

Last revised 4 August, 2023

Table of Contents

The outflow is not rolled back if the packet times out or if the acknowledgement is not successful.	2
Description	2
Problem Scenarios	3
Severity	3
Fix/Recommendation	4
Inconsistencies in Autopilot module UT cases	5
Description	5
Problem Scenarios	5
Severity	6
Fix/Recommendation	6
Denom cannot be manually blacklisted	7
Description	7
Severity	7
Fix/Recommendation	7
Channels with rate limiters could potentially have bottlenecks.....	8
Description	8
Problem Scenarios	8
Severity	8
Fix/Recommendation	9
Delay between submitting a request (proposal) and when transaction actually occurs	10
Description	10
Severity	10
Fix/Recommendation	10
Possible discrepancy between actual rate limit period and DurationHours.....	11
Description	11
Severity	12
Fix/Recommendation	12

Title	Type	Severity	Impact	Exploitability	Issue
Inconsistencies in Autopilot module UT cases	IMPLEMENTATION	0 INFORMATIONAL	0 NONE	0 NONE	
Denom cannot be manually blacklisted	IMPLEMENTATION	0 INFORMATIONAL	2 MEDIUM	0 NONE	
Channels with rate limiters could potentially have bottlenecks.	IMPLEMENTATION	0 INFORMATIONAL	0 NONE	1 LOW	
Delay between submitting a request (proposal) and when transaction actually occurs	IMPLEMENTATION	0 INFORMATIONAL	0 NONE	1 LOW	
Possible discrepancy between actual rate limit period and DurationHours	IMPLEMENTATION	0 INFORMATIONAL	0 NONE	1 LOW	
The outflow is not rolled back if the packet times out or if the acknowledgement is not successful.	IMPLEMENTATION	3 HIGH	2 MEDIUM	3 HIGH	

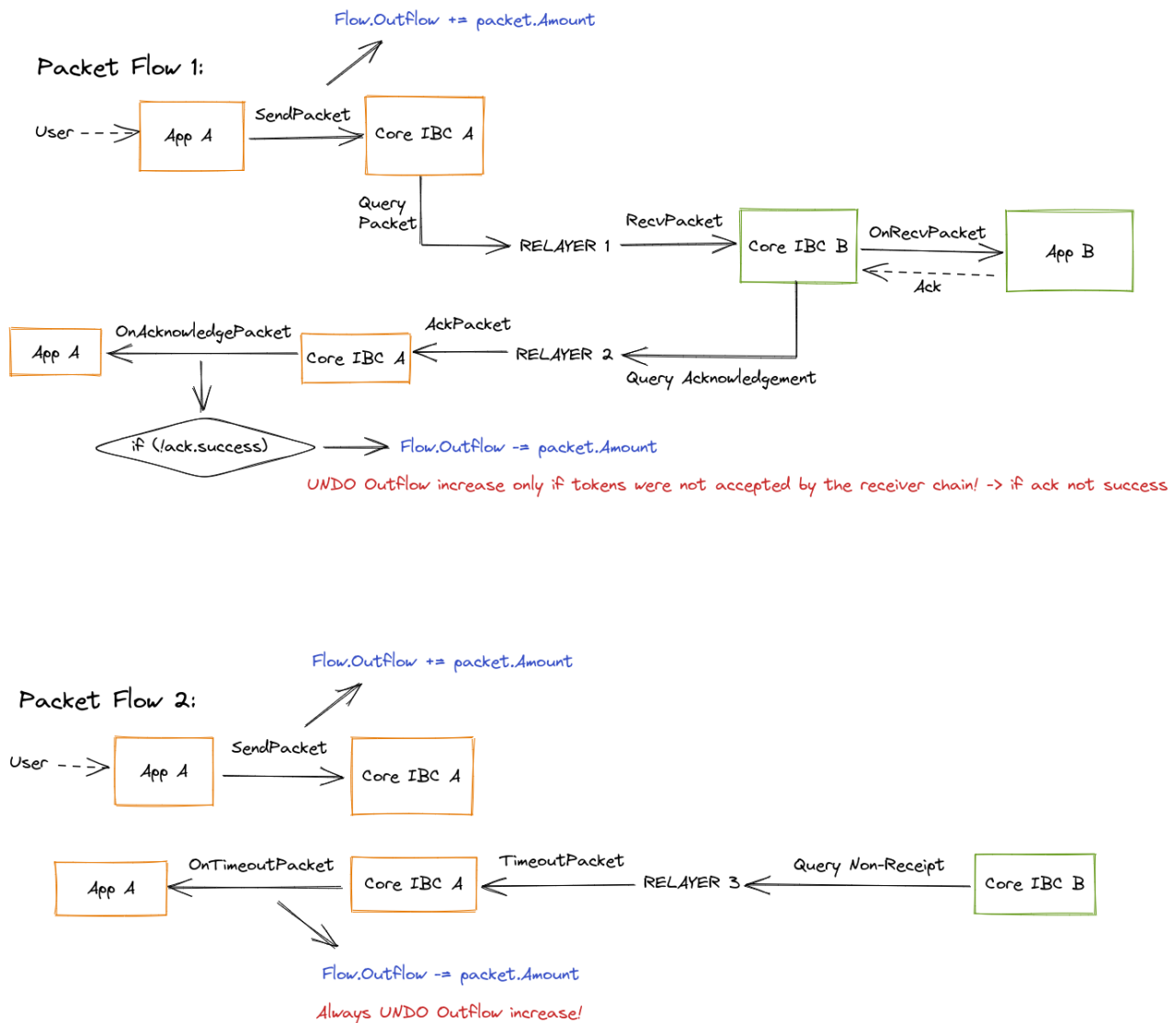
The outflow is not rolled back if the packet times out or if the acknowledgement is not successful.

Title	The outflow is not rolled back if the packet times out or if the acknowledgement is not successful.
Project	Stride: Rate Limiter & Autopilot
Type	IMPLEMENTATION
Severity	3 HIGH
Impact	2 MEDIUM
Exploitability	3 HIGH
Issue	

Description

The `ratelimit` module is implemented as IBC Middleware around the `transfer` module. Before the packet is sent, the 'SendRateLimitedPacket' middleware implementation is called. If the rate limit hasn't been exceeded, `Flow.Outflow` is updated. Normally, the `Flow.Outflow` is always either incremented or reset to zero (after each `Quota.DurationHours`).

Problem Scenarios



As can be seen from the diagram, there are two flows where the value of the outflow can go wrong:

1. `OnAcknowledgePacket` fails meaning that tokens sent from Stride are not received on another chain, and
2. `OnTimeoutPacket` happens meaning the same thing as the previous.

If a client sets a timeout, he/she could set it very low causing timeouts for a bunch of packets. This directly implies halting the chain for all outflows.

Severity

Exploitability is considered **High**, as a malicious user could easily congest a channel in one direction. The attacker is not discouraged from this behavior because they can get back the escrowed funds, regardless of whether the packet timeouts or the acknowledgment fails (due to the refund logic of the transfer app).

The impact is **Medium**, as this misbehavior could be applied to many chains that Stride communicates with, but the consequences are not severe. While there may be some temporal or unexpected packet delays when Stride sends

tokens to other chains via the transfer module, it won't affect clients from sending tokens to Stride (e.g., depositing uatoms).

Severity = **High**

Fix/Recommendation

In both cases what needs to be done is undo the `Flow.Outflow` by subtracting the amount sent. In other words, to implement something similar to the following function:

```
func undoSend(packet: Packet) {  
    FungibleTokenPacketData data = packet.GetData()  
    denom = ParseDenomFromSendPacket(data)  
    channelId = packet.GetSourceChannel()  
    rateLimiter = GetRateLimit(ctx, denom, channelId)  
    Flow.Outflow -= data.amount // critical part  
    SetRateLimit(ctx, rateLimiter)  
}
```

Inconsistencies in Autopilot module UT cases

Title	Inconsistencies in Autopilot module UT cases
Project	Stride: Rate Limiter & Autopilot
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	0 NONE
Issue	

Description

Autopilot module currently has 8 unit test cases for the automatic liquid staking process. [Test cases](#) (an exhaustive list) are:

1. param `Active=false` => don't liquid stake
2. try liquid stake native token (e.g. `ustrd`) => don't liquid stake
3. all inputs OK => do liquid stake
4. use a different channel for host denom transfer than expected => don't liquid stake
5. all inputs OK and correct memo liquid stake string => do liquid stake
6. only transfer without liquid stake (no functional part) => don't liquid stake
7. receiver has an invalid address - 1st part of the string (before `|`) => don't liquid stake
8. receiver has invalid liquid stake string - 2nd part of the string (after `|`) => don't liquid stake

Problem Scenarios

However, the 2nd and 8th test cases don't have the correct setup. The weird thing is that all tests pass, and the reason why is because all negative tests assert `NewErrorAcknowledgement` which is general assert on some error.

The intention of the 2nd test case is to fail automatic liquid staking process if someone tries to liquid stake Stride native tokens (e.g. `ustrd`). Nevertheless, `strdIbcDenom` is improperly created:

```
strdDenom := "ustrd"
prefixedDenom = transfertypes.GetPrefixedDenom(packet.GetSourcePort(),
packet.GetSourceChannel(), strdDenom)
strdIbcDenom := transfertypes.ParseDenomTrace(prefixedDenom).IBCDenom()
```

By doing so, [part](#) of the code that is supposed to be triggered by the test is skipped:

```
// In this case, we can't process a liquid staking transaction, because we're dealing
with STRD tokens
```

```

if transfertypes.ReceiverChainIsSource(packet.GetSourcePort(),
packet.GetSourceChannel(), newData.Denom) {
    return channeltypes.NewErrorAcknowledgement(errors.New("the native token is not
supported for liquid staking"))
}

```

Regarding 8th test the problem is it's in fact copy-pasted 7th test case. While it should test for improper functional part of `Receiver` field it passes because the `Receiver`'s address is invalid:

```
Receiver: "xxx|stakeibc/LiquidStake"
```

Severity

Exploitability = **None**

Impact = **None**

Severity = **Informational**

Fix/Recommendation

For the 2nd case, `strdIbcDenom` should be represented by the form `{transfer}/{source channel-id}/{ustrd}` and not in IBC hashed form.

For the 8th case, replace the functional part of the `Receiver` with something different than `stakeibc/LiquidStake`.

To avoid these hidden problems, try to always assert particular errors in the negative test scenarios. E.g. in the 2nd case, the assert should be `NewErrorAcknowledgement(errors.New("the native token is not supported for liquid staking"))`, not just general `NewErrorAcknowledgement`.

Denom cannot be manually blacklisted

Title	Denom cannot be manually blacklisted
Project	Stride: Rate Limiter & Autopilot
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	2 MEDIUM
Exploitability	0 NONE
Issue	

Description

Currently, denom can only be blacklisted automatically if the associated host zone is stopped. This is the general decision to remove admin functions.

However, this doesn't allow for an immediate response if suspicious scenarios associated with a host zone are detected.

Severity

Exploitability = **None**. With the current implementation, we have not been able to determine how the issue can be triggered.

Impact = **Medium**. Currently, a given host zone is only halted if the redemption rate is not within security limits. This leaves room for potential exploits in a single chain that can only be overcome by stopping the entire Stride chain (which breaks the isolation property). Assuming a rate limiter is in place, the impact is not **High**.

Severity = **Informational**

Fix/Recommendation

Blacklisting a denom via Multisig is an alternative way to handle extreme scenarios where a host zone is not responding to a protocol, and the security of the system is at risk. However, it is important to weigh the tradeoff between security and perception when considering this approach.

Channels with rate limiters could potentially have bottlenecks.

Title	Channels with rate limiters could potentially have bottlenecks.
Project	Stride: Rate Limiter & Autopilot
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	1 LOW
Issue	

Description

Problem Scenarios

Transferring amounts close to the rate limit could create a bottleneck in one flow direction, potentially blocking other transfers in the same direction. This is especially true for one-shot transfers.

Example 1:

If the maximum inflow or outflow rate is set at 10%, and someone transfers 9.99% of that limit in a single transaction, it could potentially lead to reduced or completely halted traffic in one direction of the channel. An attacker could then wait and execute another transaction in the opposite direction, also transferring 9.99% of the limit, thus potentially halting traffic in the next rate limit period as well.

Example 2:

It's important to note that one-shot transfers are not the only concern when it comes to potential disruptions in traffic flow due to rate limits. An attacker could also split transactions and send them in a queue, achieving the same goal as in the previous example.

Example 3:

When setting rate limits between Stride and Host Zones, it's essential to consider the following scenarios. First, every 6 hours, Stride sweeps coins to Delegation ICA, which could result in a large, one-shot transfer that could potentially break the outflow rate limit. Second, the inflow rate could also be endangered due to constant deposits of native tokens before they are moved back to the Host Zone for staking. It appears that the flows between the zones are discrete rather than alternating. In other words, there are periods of either inflows or outflows rather than a continuous alternating flow. This behavior could lead to rate limit triggers more frequently than expected.

Severity

Impact: **None**

Exploitability: **Low**

Severity: **Informational**

Fix/Recommendation

While it is possible that someone could attempt to exploit this scenario, it seems unlikely that many entities will have an amount of tokens close to the rate limit at any given time. However, preventing this scenario would not be a simple task. Limiting the amount of individual transfers alone would not solve the potential issue since an attacker could split a transaction into smaller ones. It may be effective to limit the number of transactions per user, along with the amount of each transaction, within a specific window of time. This option should be considered only if this scenario becomes prevalent.

Delay between submitting a request (proposal) and when transaction actually occurs

Title	Delay between submitting a request (proposal) and when transaction actually occurs
Project	Stride: Rate Limiter & Autopilot
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	1 LOW
Issue	

Description

It is possible for a proposal for an update or reset to be submitted during a rate limit period, but not executed until the following period. Currently, the government period is set at 3 days, while the rate limiter window can be shorter or longer.

Example:

When an update/reset proposal is submitted (epoch 1), the actual execution may occur in a later epoch (epoch+n), by which time the rate limit parameters may have already been automatically reset. This can result in unintended consequences and should be taken into account when designing the update/reset mechanism.

Severity

Impact: **None**

Exploitability: **Low**

Severity: **Informational**

Fix/Recommendation

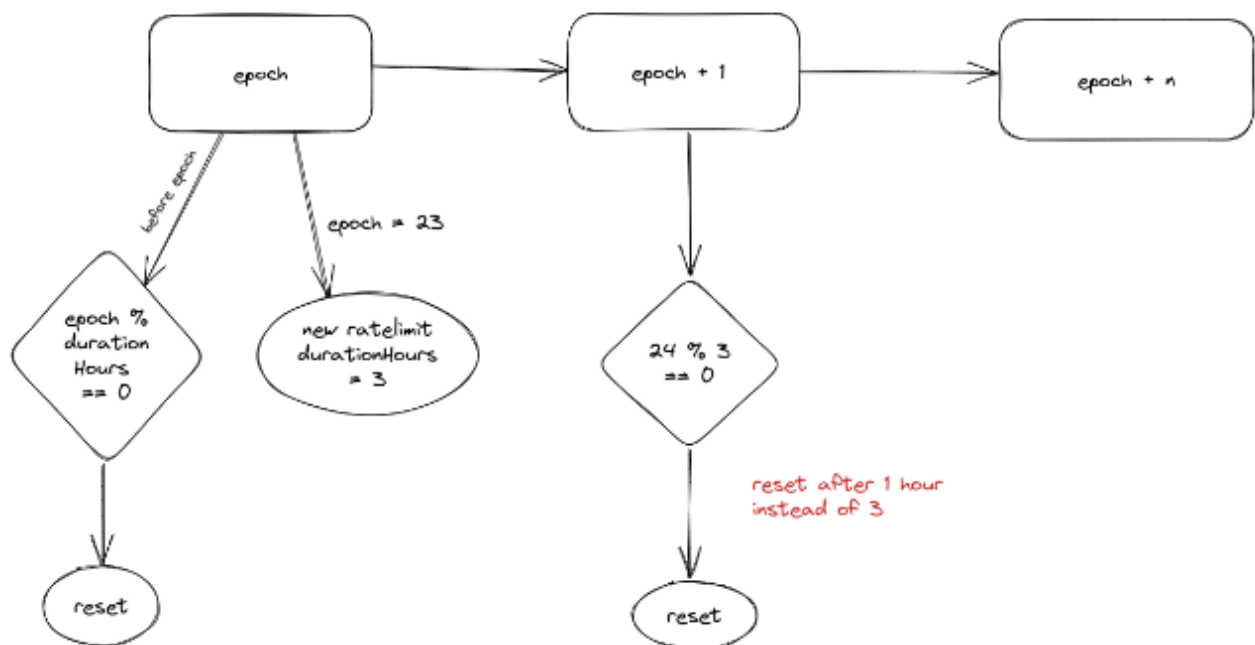
While this issue may not be exploitable, it can lead to unnecessary executions and inconsistencies. To mitigate this, consider removing the reset functionality from government methods unless there are significant use cases, such as epochs taking longer than the government period. It may also be useful to allow for the rate limit to be reset faster than the government period. The key is to ensure that the reset window is long enough to enable monitoring and action when the rate limit is tripped.

Possible discrepancy between actual rate limit period and DurationHours

Title	Possible discrepancy between actual rate limit period and DurationHours
Project	Stride: Rate Limiter & Autopilot
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	1 LOW
Issue	

Description

When launching the rate limiter for the first time or updating it, there may be a discrepancy between the epoch discrete time and the quota's `DurationHours`, which could cause the time before the reset to be less than expected.



Example:

If a new rate limit with `DurationHours` of 3 is created in epoch number 23, it will reset in the next epoch ($24 \% 3 == 0$), causing a rate limit reset after 1 hour instead of 3. However, rate limiters with the same `DurationHours` will be in sync after the first reset, meaning they will behave as expected. While there is a short time window where they may behave unexpectedly, this issue will resolve itself after the first reset.

Severity

Impact: **None**. There is no noticeable impact from shortening the initial time window size of the rate limiter.

Exploitability: The likelihood of this behavior being triggered through a government proposal is **Low**.

Severity: This issue is purely **Informational** and does not pose a significant threat.

Fix/Recommendation

This issue doesn't seem to be exploitable, but a possible reset way before expected could lead to inconsistencies, especially when tracking inflow and outflow values. Probably, if the rate limit duration is on an hourly basis it would be negligible but if the duration is on weekly basis it might have unexpected behavior for the first time (or after it's updated) the rate limiter is launched.